

“BorrowNet – Smart Campus Resource Sharing Network”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

Y. MOHANA PRIYA

2520090176

V. VISHNAVI

2520090117

Under the Esteemed Guidance of

Faculty Guide

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“BorrowNet – Smart Campus Resource Sharing Network”** is a record of bonafide work of **Y. MOHANA PRIYA – 2520090176, V. VISHNAVI – 2520090117** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

Y. MOHANA PRIYA – 2520090176
V. VISHNAVI – 2520090117

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **“BorrowNet – Smart Campus Resource Sharing Network”** is a bonafide work done and submitted by **Y. MOHANA PRIYA – 2520090176, V. VISHNAVI – 2520090117** in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, at KLH University during 2026–2027, Trimester 4.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The successful completion of this project would not have been possible without the timely help and guidance of many people. We wish to express our sincere thanks to everyone who assisted us, in one way or another, in completing this project.

We are deeply grateful to our Course Coordinator and to our project guide, Department of Computer Science and Information Technology, for their tremendous support, encouragement and guidance throughout this project, which cannot be expressed in words.

We express our gratitude to Dr. Kaja Shareef, Head of the Department of Computer Science and Information Technology, for providing us with the facilities, ways and means by which we were able to complete this project-based lab.

We thank all the teaching and non-teaching staff members who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

Y. MOHANA PRIYA – 2520090176

V. VISHNAVI – 2520090117

Abstract

Students often need items such as calculators, books, chargers, lab equipment, cameras, and project components for a short period. However, buying these items for temporary use is expensive, while existing sharing methods are informal and lack proper tracking, trust, and accountability. BorrowNet is a smart campus resource-sharing network that allows students to lend and borrow items within their college community. The system provides a centralized backend for managing users, items, availability, borrowing requests, reservations, returns, and item conditions.

The proposed solution uses a backend-driven approach to solve common resource-sharing problems. Students can list items they are willing to lend, search for available resources, request items, and reserve them for specific periods. The system prevents overlapping reservations, maintains item-condition history, calculates user trust scores, and applies penalties for late returns or damaged items. Automated notifications can remind users about upcoming and overdue returns. An admin module can handle reports, disputes, and user moderation.

BorrowNet promotes affordable resource utilization, reduces unnecessary purchases, and encourages collaboration among students. The project demonstrates important backend concepts including authentication, authorization, database management, REST APIs, reservation conflict handling, transactions, notifications, and business-rule processing.

The application is designed with a React-based frontend, a Java Spring Boot backend exposing RESTful APIs secured with JSON Web Tokens, and a MySQL relational database. The relational model of ten tables (users, items, categories, item images, borrow requests, reservations, transactions, ratings, notifications and penalties) has been designed with primary keys, foreign keys and validation constraints. At the Review 2 stage, the system architecture, borrowing lifecycle, ER diagram, relational schema and a frontend UI prototype have been completed; backend API development and database integration form the next implementation stage.

Index

S.No.	Chapters	Topics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience 1.7 Existing vs. Proposed System	7
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	9
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security	10
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	12
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Sample Records 5.4 Design Decisions and Assumptions	13
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	18
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	21
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Status	23
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	25
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations	26
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	27
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	28
13	References	Documentation, tools, and resources used	29
14	Appendices	A – Sample Code Snippets B – Installation / Setup Instructions C – Default Login Credentials D – API Reference Summary E – Screenshots	30

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Students regularly need items for only a short period of time — a scientific calculator for an exam, a textbook for a single unit, a charger, a camera for a club event, a lab instrument or a handful of components for a project. Buying such items is costly and inefficient when they will be used only once or twice, while many students already own the same items and leave them unused for most of the year.

In most colleges this sharing happens informally: students ask friends, post in class groups, or send direct messages. This works only in small circles and breaks down quickly. Nobody has a clear view of which items exist or are free, two people can ask for the same item for the same dates, return dates are forgotten, and there is no record of an item's condition or of how reliably someone returns what they borrow.

BorrowNet was conceived to replace this informal coordination with a single, database-backed platform for the campus community. Every user, item, request, reservation, return and rating is stored in one relational database, and the backend — rather than a person cross-checking chat messages — is responsible for detecting conflicting reservations, tracking returns and item condition, and maintaining trust between users. The guiding idea of the project is *Share More. Spend Less. Connect Better.*

1.2 Problem Statement

Students often need academic resources for a short period, but purchasing items such as calculators, books, chargers, lab equipment, or project components can be costly and inefficient. Informal sharing through friends or messaging groups does not provide a centralized way to track availability, requests, reservations, returns, item condition, or user accountability.

1.3 Scope of the Project

The initial scope of BorrowNet is student-to-student resource sharing within a college community. It covers:

- Student registration, login, and authenticated access.
- Item listing, categories, search, and availability checking.
- Borrow requests, owner approval, reservations, borrowing, and return tracking.
- Trust scores, ratings, item-condition records, late-return penalties, and notifications.
- Admin moderation of users, items, reports, and disputes.

A native mobile application, QR-based pickup and return, AI-based item recommendations, and smart demand prediction are treated as future scope (see Chapter 11).

1.4 Objectives

- Create a structured platform for students to lend and borrow campus resources.
- Maintain accurate item availability and reservation information.
- Prevent conflicting reservations for the same item.
- Track borrowing, returns, item condition, ratings, and user trust.
- Improve accountability through notifications and penalty rules.
- Demonstrate authentication, authorization, REST APIs, database transactions, and business-rule processing.

1.5 Importance of the Application

By centralizing item and reservation data, BorrowNet removes guesswork from peer-to-peer sharing. Students can see what is available and when, reserve it with confidence and avoid unnecessary purchases. Owners are protected by accountability mechanisms — ratings, trust scores, condition checks and late-return penalties — which make lending a low-risk activity. Administrators gain a single place to moderate users and items and to resolve disputes. At the same time, the project is a practical demonstration of real-world backend engineering concepts.

1.6 Target Users / Audience

The system serves two roles. **Students** use it to list items they are willing to lend, search and browse resources, check availability, send borrow requests, approve or reject requests for their own items, return items, rate other users and follow their notifications and trust score. **Administrators** use it to moderate users and items, review reports, handle disputes and monitor activity across the platform.

1.7 Existing System vs. Proposed System

The existing approach is mainly informal peer-to-peer sharing through personal contacts, class groups and direct messages. The table below compares it with the proposed BorrowNet system.

Aspect	Existing System	Proposed System (BorrowNet)
Finding items	Asking friends or groups; limited visibility	Central item listing with categories and search
Availability	Manual tracking; no central record	Availability and reserved periods maintained by the backend
Reservations	No reservation control; overlapping requests	Conflict detection prevents overlapping reservations
Returns and condition	Return dates and condition disputes handled by memory	Return and condition recorded in transaction history
Accountability	Limited accountability between users	Ratings, trust score and late-return / damage penalties
Communication	Scattered chat messages	Automated notifications for requests, approvals and reminders
Moderation	None	Admin module for users, items, reports and disputes

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

BorrowNet has modest hardware requirements, because both the Spring Boot backend and the MySQL database are lightweight for the dataset of a single campus. The minimum environment needed to develop and run the application is listed below.

Component	Minimum requirement
Device	Laptop / Desktop
Memory	Minimum 8 GB RAM
Processor	Intel i5 / AMD Ryzen 5 or equivalent
Storage	256 GB SSD or higher
Network	Stable internet connection

2.2 Software Requirements

Category	Software
Programming language	Java (a supported JDK release)
Backend framework	Spring Boot
Frontend	React.js with HTML5, CSS3 and JavaScript
Database	MySQL
Authentication	JWT (JSON Web Token) Authentication
API style	REST APIs
Build tool	Maven
Browser	A modern web browser (Chrome, Edge or Firefox)

2.3 Development Tools and Frameworks

React.js was chosen for the single-page frontend; Java with the Spring Boot framework for the backend REST API, built and managed with Maven; and MySQL as the relational database. Postman is used for API testing, Git and GitHub for version control and collaboration, IntelliJ IDEA and Visual Studio Code as development environments, and MySQL Workbench for designing, inspecting and verifying the database schema and data. Figma was used for design and prototyping of the user interface screens.

Tool	Purpose
IntelliJ IDEA / VS Code	Backend and frontend development
MySQL Workbench	Schema design, queries, data verification
Postman	API testing and documentation
Git & GitHub	Version control and team collaboration
Figma	UI design and prototyping

CHAPTER 3: TECHNOLOGY STACK

Area	Technology / Tool
Frontend	React.js, HTML5, CSS3, JavaScript
Backend	Java, Spring Boot
Database	MySQL
Authentication	JWT (JSON Web Token)
API	RESTful APIs
Testing	Postman
Version Control	Git & GitHub
Development	IntelliJ IDEA / VS Code
Build Tool	Maven
Database Tool	MySQL Workbench
Deployment	Render / Railway
Design & Prototyping	Figma

3.1 Frontend

The frontend is a single-page application built with React.js, HTML5, CSS3 and JavaScript. It provides a home page, login and registration, a student dashboard, item browsing and details, and borrow-request forms. The frontend never accesses the database directly; all data comes from the backend REST API. The screens were first designed and prototyped in Figma (see Appendix E).

3.2 Backend

The backend is a REST API built with Java and Spring Boot and managed with Maven. It is organized by responsibility — request handling, business logic and data access — so that rules such as availability checking, conflict detection, trust scoring and penalties live in one clearly defined layer instead of being scattered across the application.

3.3 Database

MySQL stores all persistent data across ten related tables (user, item, category, item_image, borrow_request, reservation, transaction, rating, notification and penalty). Referential integrity is enforced with foreign keys, uniqueness with UNIQUE constraints (for example, one account per email address) and data validity with constraints such as the rating range of 1 to 5. MySQL Workbench is used to design and verify the schema.

3.4 Version Control

Git is used for version control, and the project source code is hosted on GitHub so that both team members can collaborate, track changes and maintain a history of the project.

3.5 Authentication and Security

Login sessions are stateless and use JSON Web Tokens (JWT). After a successful login the server issues a signed token that identifies the user and role; the backend verifies this token on every protected request. Role-based authorization separates student functions from admin functions, so a student account cannot reach admin-only endpoints. Passwords are to be stored only as salted hashes — never as plain text.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

BorrowNet follows a client–server architecture organised into three tiers that separate the user interface, the business logic and the persisted data. Figure 4.1 shows the layers and how a request flows between them.

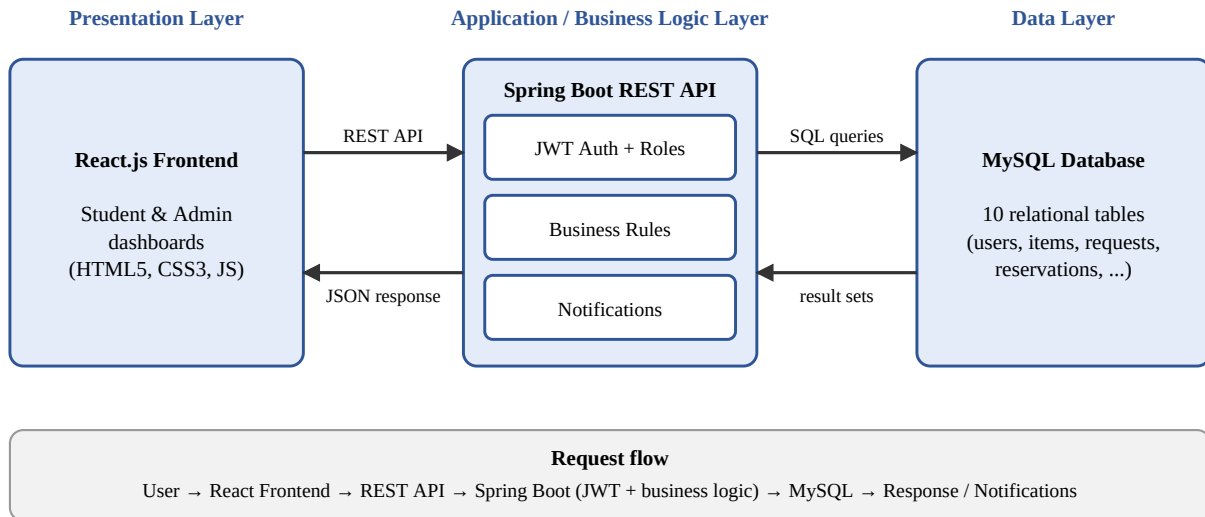


Figure 4.1: High-level architecture of BorrowNet

4.2 Description of Each Layer

Presentation Layer. The React.js single-page application is the only part the user sees. It sends every request to the backend as a REST API call and renders the response. It never talks to MySQL directly.

Application Layer. The Spring Boot REST API receives the request and passes it through two internal layers:

- **Security layer** — JWT authentication verifies who is calling, and role-based authorization decides what that user is allowed to do.
- **Business layer** — validates the request and applies the platform rules: availability checks, reservation conflict detection, trust score calculation, penalty rules and notification generation. Multi-step operations run inside database transactions.

Data Layer. The MySQL database stores users, items, requests, reservations and transactions along with ratings, notifications and penalties, and enforces integrity through keys and constraints. The result of each operation flows back through the REST API to the frontend as a response, and relevant notifications are generated for the users concerned.

4.3 Deployment Architecture

During development and demonstration, the three tiers run as separate local processes on one machine: the React development server, the Spring Boot application and a local MySQL instance. For hosting beyond the local environment, Render or Railway have been identified as deployment targets (Chapter 9). Containerization and CI/CD pipelines are not part of the current scope and are discussed under future enhancements.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The relational schema is designed to be normalized: each fact is stored in exactly one place, and every relationship between entities is expressed through a foreign key rather than by duplicating data across tables. Every table defines its own invariants in the database — using PRIMARY KEY, FOREIGN KEY, UNIQUE and validation constraints — so that invalid data (for example a duplicate email or a rating outside 1 to 5) is rejected at the database level even if the application code were to attempt to insert it.

5.2 ER Diagram / Database Schema

The database consists of ten related entities, shown in Figure 5.1: USER, ITEM, CATEGORY, ITEM_IMAGE, BORROW_REQUEST, RESERVATION, TRANSACTION, RATING, NOTIFICATION and PENALTY.

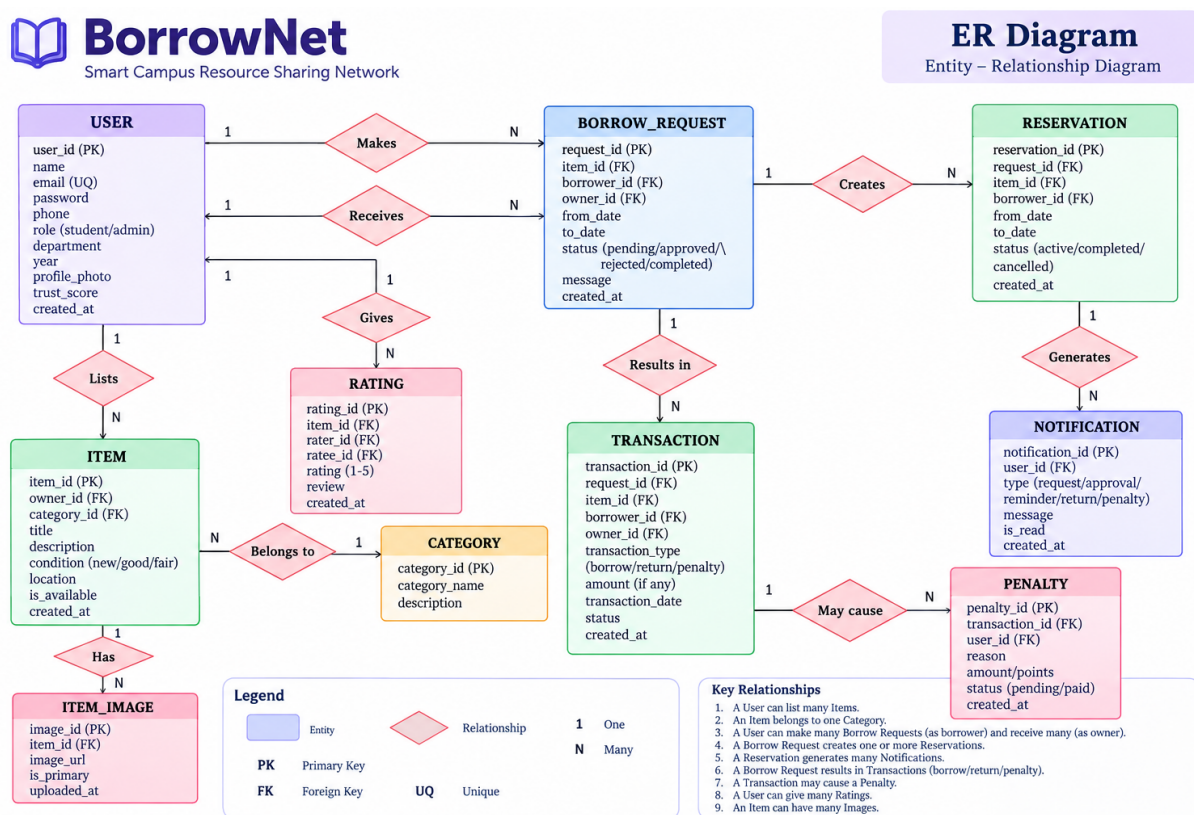


Figure 5.1: Entity–Relationship diagram of BorrowNet

The key relationships in the model are:

1. A User can list many Items.
2. An Item belongs to one Category and can have many Item Images.
3. A User can make many Borrow Requests (as borrower) and receive many (as owner).
4. A Borrow Request creates one or more Reservations.
5. A Reservation generates Notifications.
6. A Borrow Request results in Transactions (borrow / return / penalty).
7. A Transaction may cause a Penalty.
8. A User can give many Ratings.

Data dictionary

The tables below describe each entity, its columns and keys, as defined in the ER diagram.

USER – identity, role and trust

Column	Key	Description
user_id	PK	Unique identifier of the user
name		Full name
email	UQ	Login email; unique per account
password		Hashed password
phone		Contact number
role		student / admin
department, year		Academic details of the student
profile_photo		Profile picture
trust_score		Score derived from ratings and penalties
created_at		Registration timestamp

CATEGORY – item categories

Column	Key	Description
category_id	PK	Unique identifier of the category
category_name		Name (e.g. Books, Electronics, Lab Equipment)
description		Short description of the category

ITEM – shared resource details

Column	Key	Description
item_id	PK	Unique identifier of the item
owner_id	FK	References USER – the lender
category_id	FK	References CATEGORY
title, description		What the item is
condition		new / good / fair
location		Where the item can be collected
is_available		Whether the owner currently offers the item
created_at		Listing timestamp

ITEM_IMAGE – photographs of an item

Column	Key	Description
image_id	PK	Unique identifier of the image
item_id	FK	References ITEM
image_url		Location of the stored image
is_primary		Marks the main image of the item
uploaded_at		Upload timestamp

BORROW_REQUEST – borrowing workflow

Column	Key	Description
request_id	PK	Unique identifier of the request
item_id	FK	References ITEM
borrower_id	FK	References USER – the requester
owner_id	FK	References USER – the item owner
from_date, to_date		Requested borrowing period
status		pending / approved / rejected / completed
message		Optional purpose or note from the borrower
created_at		Request timestamp

RESERVATION – reserved date range

Column	Key	Description
reservation_id	PK	Unique identifier of the reservation
request_id	FK	References BORROW_REQUEST
item_id	FK	References ITEM
borrower_id	FK	References USER
from_date, to_date		Reserved period used for conflict checks
status		active / completed / cancelled
created_at		Creation timestamp

TRANSACTION – borrow / return history

Column	Key	Description
transaction_id	PK	Unique identifier of the transaction
request_id	FK	References BORROW_REQUEST
item_id, borrower_id, owner_id	FK	References ITEM and USER
transaction_type		borrow / return / penalty
amount		Amount, if any
transaction_date, status		When it happened and its state
created_at		Record timestamp

RATING – feedback between users

Column	Key	Description
rating_id	PK	Unique identifier of the rating
item_id	FK	References ITEM
rater_id	FK	References USER – who gave the rating
ratee_id	FK	References USER – who received it
rating		Value from 1 to 5
review		Optional written review
created_at		Timestamp

NOTIFICATION – alerts to users

Column	Key	Description
notification_id	PK	Unique identifier of the notification
user_id	FK	References USER – the recipient
type		request / approval / reminder / return / penalty
message		Text shown to the user
is_read		Read / unread flag
created_at		Timestamp

PENALTY – accountability records

Column	Key	Description
penalty_id	PK	Unique identifier of the penalty
transaction_id	FK	References TRANSACTION
user_id	FK	References USER – the penalised user
reason		Late return, damage, etc.
amount / points		Magnitude of the penalty
status		pending / paid
created_at		Timestamp

5.3 Sample Records

The following illustrative records were prepared for prototype and database testing. Final values will come from the implemented database.

Table	Sample record
USER	252001 Vishnavi student trust score 4.8
ITEM	101 Scientific Calculator available owner 252001
BORROW_REQUEST	5001 item 101 borrower 2520090176 pending
RESERVATION	7001 item 101 10–12 May active

5.4 Design Decisions and Assumptions

The following decisions were taken explicitly during design, and are recorded here rather than left implicit in the code:

- **Single source of truth for availability.** Only active RESERVATION rows are consulted when checking whether an item is free for a period. BORROW_REQUEST records the workflow (who asked, who approved), and TRANSACTION keeps the history of borrow, return and penalty events.
- **Two kinds of availability.** The is_available flag on ITEM tells whether the owner currently offers the item at all; time-based availability always comes from the reservation conflict check.
- **Atomic multi-table operations.** Operations that write to several tables — such as approving a request (request status, reservation, notification) — run inside one database transaction so a failure leaves no partial state.
- **Half-open reservation periods.** A reservation period is treated as [from, to): a new reservation may start exactly when another one ends.
- **Stored trust score.** trust_score is stored on USER for fast reads and is recomputed whenever a rating or penalty is recorded.
- **Roles assigned by the server.** Public registration always creates a student account; the role is never accepted from the client.
- **History is preserved.** Items that already have borrowing history are marked unavailable rather than physically deleted.

CHAPTER 6: IMPLEMENTATION

Implementation status (Review 2): The frontend UI prototype, system architecture, borrowing lifecycle, ER diagram, relational schema and sample records are complete. Backend APIs and database integration are the next implementation stage. This chapter therefore documents the implementation design that the backend will follow, together with the current status.

Component	Status
Project abstract and Review 1 documentation	Completed
Frontend UI prototype (home, login, dashboard, browse, details, request screens)	Completed (prototype)
System architecture and borrowing lifecycle	Completed
ER diagram and relational schema (10 tables)	Completed
Sample records for database testing	Prepared
Backend APIs and database integration	Next stage
Connecting the frontend to the APIs, testing and deployment	Planned

6.1 Module-wise Implementation

The backend is organised into eight functional modules. Each module has its own REST endpoints, business logic and data access.

Authentication Module. Handles registration, login, JWT issuance and roles (student / admin). Registration always creates a student account; passwords are stored as salted hashes; a successful login returns a signed token carrying the user's ID and role.

Item Management Module. Lets owners list items with title, description, category, condition, location and images, edit or remove their listings, and lets all users search and filter items by name and category.

Borrow Request Module. A borrower requests an item for a date range with an optional message. The owner approves or rejects the request; its status moves through pending, approved / rejected and completed, and both parties can track it.

Reservation Module. Validates the requested dates and runs the conflict check against existing active reservations of the same item. A reservation is created only when the check passes, and it can be cancelled.

Transaction and Return Module. Records borrowing and return events (and penalty events) in the transaction history. At return, the item's condition is recorded and the return time is compared with the reserved end date.

Trust and Penalty Module. Stores ratings (1–5 with an optional review), computes each user's trust score from ratings and penalty history, and creates penalty records for late returns or damaged items.

Notification Module. Generates notifications for new requests, approvals and rejections, upcoming-due reminders, return and overdue alerts, and penalties. Notifications carry a read / unread flag and appear in the user's dashboard.

Admin Module. Provides moderation of users and items, review of reports, dispute handling and monitoring of activity. All admin endpoints are protected by a role check.

Core business rules

Conflict detection. A requested period conflicts with an existing active reservation of the same item if (existing_from < requested_to) AND (existing_to > requested_from). This single rule is applied wherever availability matters — the availability check shown to students and the approval of a borrow request — so an item can never be double-reserved, whichever entry point is used.

Approval workflow. The approval of a request runs as one database transaction:

1. Verify that the caller is the owner of the item and the request is still pending.
2. Re-run the conflict check against active reservations for the requested dates.
3. Set the request status to approved and insert the reservation with status active.
4. Insert a notification for the borrower.
5. Commit — if any step fails, every write is rolled back.

Return workflow. On return, the backend records a return transaction, captures the item's condition, completes the reservation, and compares the return time with the reserved end date. A late return or a reported damage creates a penalty record and a notification; after the borrower and owner rate each other, the trust scores are recomputed.

6.2 Front-end Logic

The frontend UI prototype covers the home page, login / registration, student dashboard, item browsing, item details and the borrow-request form (Appendix E). The dashboard presents the numbers a student cares about at a glance — listings, active requests, items borrowed and trust score — with quick actions for listing an item, browsing, viewing requests and messages. A sidebar gives access to Browse Items, My Requests, My Listings, Messages, Notifications and Profile.

When connected to the backend, every page will fetch its data through one central API service layer instead of hard-coding URLs. This layer will attach the user's JWT to outgoing requests and return the user to the login page whenever the server answers with 401 Unauthorized (for example after the token expires). Shared UI elements — forms, cards, status badges and notification indicators — will be written once and reused across pages so that loading, empty and error states look consistent.

6.3 API Integration

The frontend and backend communicate exclusively through RESTful JSON APIs. The planned endpoints are summarised in Appendix D. The APIs use standard HTTP status codes so the frontend can react correctly:

Code	Meaning in BorrowNet
200 / 201	Request succeeded / resource created
400	Validation error (missing field, invalid dates, rating out of range)
401	Missing, invalid or expired token
403	Authenticated but not allowed (for example a student calling an admin endpoint)
404	Requested resource does not exist
409	Business conflict (overlapping reservation, duplicate email)

6.4 Authentication and Authorization

Authentication is performed with JWT. The login endpoint verifies the credentials and returns a signed token; for every other protected route, a request filter validates the token and attaches the caller's identity and role to the request. **Authorization** works at two levels. Role checks separate student endpoints from admin endpoints (a student token receives 403 Forbidden on an admin route). Ownership checks ensure that only the owner of an item can approve or reject requests for it, and only the borrower can cancel their own request. Because the role is assigned by the server and never accepted from the registration form, a user cannot register their way into administrator access.

CHAPTER 7: FEATURES

7.1 List of Main Features

Student-facing features:

- Register and log in securely.
- List items for lending with category, condition, location and images.
- Browse, search and filter items; view full item details.
- Check item availability for a chosen date range.
- Send a borrow request and track its status (pending / approved / rejected / completed).
- Approve or reject incoming requests for one's own items.
- Return items and rate the other party; view personal trust score and ratings.
- Receive notifications for requests, approvals, reminders, returns and penalties.
- View a personal profile and dashboard.

Administrator-facing features:

- Moderate users and items.
- Review reports and handle disputes between users.
- Monitor platform activity.
- Maintain accountability across the community.

7.2 How Each Feature Works

The borrowing lifecycle that ties the features together is shown in Figure 7.1.

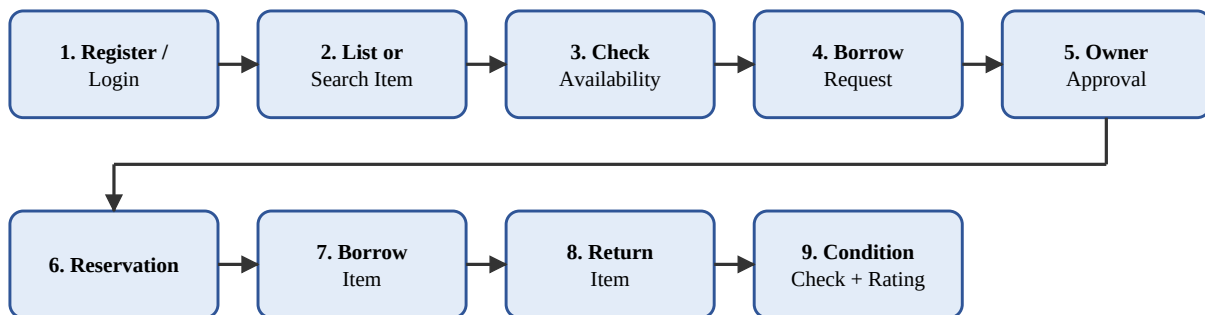


Figure 7.1: BorrowNet borrowing lifecycle — the smart backend (conflict detection, trust score, penalties, notifications and transactions) governs every step

Availability checking and overlap prevention. When a student selects an item and a date range, the backend looks for any active reservation of that item that overlaps the range using the conflict-detection rule of Chapter 6. If one exists, the request is rejected with a message naming the clashing period; if none exists, the request can proceed. A reservation that begins exactly when another ends is allowed.

Request, approval and reservation. A borrow request starts as pending and the owner is notified. When the owner approves it, the backend re-checks for conflicts and — in a single transaction — marks the request approved, creates the active reservation and notifies the borrower. If the owner rejects it, the borrower is notified and no reservation is created.

Borrowing, return and condition tracking. Handover and return are recorded as transactions. At return, the item's condition is checked and recorded, so that changes in condition (for example from good to fair) are traceable to a specific borrowing.

Ratings and trust score. After a return, users can rate each other from 1 to 5 with an optional review. A user's trust score is derived from the ratings received and from penalties applied, and is shown on the dashboard so that owners can decide whom to lend to.

Late-return and damage penalties. If an item is returned after the reserved end date, or damage is reported at the condition check, the penalty engine creates a penalty record with a reason, an amount or points value and a status (pending / paid), and the user is notified. Penalties feed into the trust score.

Notifications. The backend creates notifications for new requests, approvals and rejections, upcoming-due reminders, return confirmations, overdue alerts and penalties. They are stored with a read flag and shown through the notification bell on the dashboard.

Admin moderation. Administrators can review users and items, act on reports and resolve disputes. Their actions are available only through admin-only endpoints guarded by the role check.

CHAPTER 8: TESTING

Testing status: Backend implementation is the next stage, so no test results have been recorded yet. This chapter defines the testing strategy and the test cases that will be executed; the Status column will be updated to Pass / Fail as each case is run.

8.1 Postman API Testing

A Postman collection will be built for all REST endpoints, organised into folders matching the backend modules (Auth, Items, Borrow Requests, Reservations, Transactions, Ratings, Notifications and Admin). The login requests will include a small test script that saves the returned JWT into a collection variable, and requests such as Create Item, Create Request, Approve Request and Return Item will be chained using variables, so that a complete borrowing workflow can be exercised without copying any ID or token by hand.

8.2 Integration Testing

Each backend module will be tested end to end against a real, running MySQL database as soon as it is built, rather than in isolation with mocked data, before the next module is built on top of it. As a final check, the database will be dropped and rebuilt from the schema and seed scripts, and the full set of test cases will be re-run on the fresh installation to confirm that a first-time setup works correctly.

8.3 Tools Used for Testing

- Postman — manual and scripted API testing.
- MySQL Workbench — direct verification of table contents, constraints and transaction behaviour.
- Web browser — manual testing of the React frontend once it is connected to the backend.

8.4 Test Cases and Status

The planned test cases below target the rules that matter most to BorrowNet: security, conflict detection, transactions, penalties and validation.

#	Test case	Expected result	Status
1	Login with a wrong password	401; login rejected	Planned
2	Register with an email that already exists	409; friendly duplicate-email message	Planned
3	Call a protected endpoint without a token	401 Unauthorized	Planned
4	Student token calls an admin-only endpoint	403 Forbidden	Planned
5	Create an item with a missing required field	400; validation message	Planned
6	Create a request whose from_date is after to_date	400; rejected before reaching the database	Planned
7	Request an item the owner has marked unavailable	Rejected with a clear message	Planned
8	Approve a request that overlaps an active reservation	409; message naming the clashing period	Planned

#	Test case	Expected result	Status
9	Reserve a slot that starts exactly when another ends	No conflict (back-to-back allowed)	Planned
10	Approve a request (normal case)	Request approved, reservation active, notification created — all in one transaction	Planned
11	Force a failure during approval	Full rollback; no partial request, reservation or notification rows	Planned
12	Two users try to reserve the same item for the same dates at once	Exactly one succeeds; the other gets 409	Planned
13	A non-owner tries to approve a request	403 Forbidden	Planned
14	Return an item after the reserved end date	Return transaction recorded, penalty created, notification sent	Planned
15	Return an item reporting damage	Condition recorded and penalty applied per rule	Planned
16	Submit a rating outside the range 1–5	400; rejected	Planned
17	Submit a valid rating	Rating stored; ratee's trust score recomputed	Planned
18	List notifications and mark one as read	is_read updated; unread count decreases	Planned

CHAPTER 9: DEPLOYMENT

Deployment status: The application is currently developed and demonstrated locally. The steps below describe the planned local setup and the intended hosting approach; they will be confirmed once the backend is complete.

9.1 Steps to Deploy

1. Create the database by running the schema script and then the seed script in MySQL (for example from MySQL Workbench).
2. Configure the backend with the database connection details and the JWT secret (see 9.2).
3. Start the backend with Maven: `mvn spring-boot:run` (Spring Boot listens on port 8080 by default).
4. Install the frontend dependencies and start the React development server.
5. Open the application in a browser and log in with a seeded account.

9.2 Environment Configuration

All environment-specific settings are kept outside the source code, in application configuration or environment variables that are never committed to version control. They cover the database connection (URL, user, password), the JWT signing secret and token expiry, and the frontend origin allowed by CORS. A template with placeholder values is committed instead, so the required settings are documented without exposing real credentials.

9.3 Hosting of Frontend and Backend

For the current submission, the frontend, the backend and the database run locally, which is appropriate for development and in-class demonstration. Render and Railway have been identified as the deployment platforms for hosting the backend and the frontend; hosting configuration will be finalised after backend implementation and testing. A production deployment is not required for this stage of the course.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

The following challenges have been identified during analysis and design and will need careful handling during implementation:

- Preventing two users from reserving the same item for overlapping dates.
- Keeping item availability consistent after approval, borrowing and return.
- Implementing secure role-based access for students and admins.
- Handling late returns, penalties and item-condition changes.
- Testing edge cases and maintaining consistent transaction records.

10.2 Solutions Applied

Overlapping reservations. A single, consistently applied overlap rule is used for every availability check and for request approval, with RESERVATION as the only table that conflict checks read. The check is repeated at approval time, inside the transaction, so that a stale check cannot cause a double booking.

Consistent availability. Reservation and request state changes are performed together inside database transactions, and the `is_available` flag (owner's offer) is kept separate from time-based availability (reservations), so one cannot silently contradict the other.

Secure access. JWT authentication with role checks on every protected route, ownership checks on request approval and cancellation, and server-side role assignment during registration.

Penalties and condition changes. A rule-based penalty engine runs when an item is returned, comparing the return time with the reserved end date and checking the reported condition; the outcome is stored as a penalty record linked to the transaction.

Edge cases. Edge cases — back-to-back reservations, concurrent approvals, rollback after a mid-operation failure, invalid dates — are included explicitly in the test plan of Chapter 8.

10.3 Current Limitations

- The backend APIs and database integration are not yet implemented; the frontend screens are a prototype and not yet connected to live data.
- Sample records are illustrative; test results are not yet available.
- The current schema covers users, items and transactions; dedicated tables for reports and disputes may be added during implementation of the admin module.
- No email verification or password-reset flow is planned for the first version.
- Notifications are generated inside the application; real-time push notifications are future work.
- The system has not yet been deployed to a public hosting platform.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- **Mobile application** — Android / iOS access for faster campus sharing.
- **QR-based pickup and return** — scan an item to confirm handover and return.
- **AI-based item recommendations** — suggest useful resources based on user needs and history.
- **Smart demand prediction** — identify frequently requested resources.
- **Real-time notifications** — push reminders for approvals, due dates and returns.
- **Analytics** — track sharing activity, popular items and system usage.

11.2 Possible Integrations or Optimizations

- Email or push-notification services for reminders and overdue alerts.
- Caching of frequently requested item-availability queries to reduce repeated database load.
- Containerizing the application with Docker for a consistent, reproducible deployment environment.
- An automated backend test suite (for example JUnit) to complement the Postman-based testing, so regressions are caught automatically.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

BorrowNet addresses a simple but common campus problem: students need resources for a short time, and informal sharing offers no tracking, trust or accountability. The project proposes a centralized, backend-driven platform in which students list, search, request, reserve, borrow and return items, while the backend enforces the rules — preventing overlapping reservations, tracking item condition, maintaining trust scores, applying penalties and generating notifications — and an admin module provides moderation.

12.2 What Was Achieved

At the Review 2 stage the following have been completed:

- Project abstract and Review 1 documentation.
- A frontend UI prototype covering the home, login, dashboard, browse, item-detail and request screens.
- System architecture and the complete borrowing lifecycle.
- An ER diagram and relational schema of ten tables, with sample records prepared for database testing.

Backend APIs and database integration are the next implementation stage, followed by testing against a real MySQL database and deployment.

12.3 Skills Learned

Working on this project provides practical experience in relational database design and normalization; defining constraints at the database layer; designing REST APIs with meaningful HTTP status codes; JWT-based authentication with role-based authorization; managing multi-table transactions and reservation conflict handling; UI design and prototyping with Figma; API testing with Postman; and team collaboration with Git and GitHub.

CHAPTER 13: REFERENCES

- Spring Boot — Official Documentation, <https://spring.io/projects/spring-boot>
- React — Official Documentation, <https://react.dev>
- Apache Maven — Official Documentation, <https://maven.apache.org>
- MySQL Reference Manual, <https://dev.mysql.com/doc/>
- JSON Web Token (RFC 7519), <https://datatracker.ietf.org/doc/html/rfc7519>
- Postman Learning Center, <https://learning.postman.com>
- Git — Documentation, <https://git-scm.com/doc>
- GitHub Docs, <https://docs.github.com>
- Figma — Help Center, <https://help.figma.com>
- Render — Documentation, <https://render.com/docs>
- Railway — Documentation, <https://docs.railway.com>
- MDN Web Docs — JavaScript and Web APIs reference, <https://developer.mozilla.org>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippets

Illustrative design sketches of the two most important backend rules. Final code will follow the same logic once the backend is implemented.

The overlap rule used for availability search and request approval:

```
-- Any returned row means the requested period conflicts.
SELECT r.reservation_id, r.from_date, r.to_date
FROM   reservation r
WHERE  r.item_id    = ?
      AND r.status   = 'active'
      AND r.from_date < ? -- requested to_date
      AND r.to_date  > ?; -- requested from_date
```

Approving a borrow request inside one transaction:

```
@Transactional
public Reservation approveRequest(Long requestId, Long ownerId) {
    BorrowRequest req = requestRepo.findByIdForUpdate(requestId);
    assertOwner(req, ownerId);
    assertStatus(req, Status.PENDING);
    if (reservationRepo.existsOverlap(req.getItemId(),
                                     req.getFromDate(), req.getToDate()))
        throw new ConflictException("Item already reserved for this period");
    req.setStatus(Status.APPROVED);
    Reservation res = reservationRepo.save(Reservation.from(req, Status.ACTIVE));
    notificationService.notify(req.getBorrowerId(), "approval",
                              "Your borrow request was approved");
    return res; // any exception above rolls back all writes
}
```

Appendix B: Installation and Setup Instructions (planned)

- Install the JDK, Maven, MySQL, Node.js and Git.
- Clone the project repository from GitHub.
- Run the schema script and then the seed script in MySQL.
- Set the database connection details and JWT secret in the backend configuration.
- Backend: `mvn spring-boot:run`
- Frontend: install dependencies and start the React development server.
- Open the application in a browser.

Appendix C: Default Login Credentials

Seed accounts (one administrator and several student accounts) will be created by the seed script when the backend is implemented, and will be listed here with their email addresses and the default password. No credentials exist at the current stage.

Appendix D: API Reference Summary (proposed)

Proposed REST endpoints by module; the final request and response examples will be provided in the accompanying Postman collection.

Resource	Proposed endpoints
Auth	POST /api/auth/register, POST /api/auth/login, GET /api/auth/me
Items	GET /api/items, GET /api/items/{id}, GET /api/items/{id}/availability, POST /api/items, PUT /api/items/{id}, DELETE /api/items/{id}
Categories	GET /api/categories
Borrow Requests	POST /api/requests, GET /api/requests/my, GET /api/requests/incoming, PUT /api/requests/{id}/approve, PUT /api/requests/{id}/reject
Reservations	GET /api/reservations/my, PUT /api/reservations/{id}/cancel
Transactions	POST /api/transactions/borrow, POST /api/transactions/return, GET /api/transactions/my
Ratings	POST /api/ratings, GET /api/users/{id}/ratings
Notifications	GET /api/notifications, PUT /api/notifications/{id}/read
Admin	GET /api/admin/users, PUT /api/admin/users/{id}/status, GET /api/admin/items, GET /api/admin/activity

Appendix E: Screenshots

The following screens are the frontend UI prototype prepared for Review 2. The final implementation will be connected to the backend APIs.

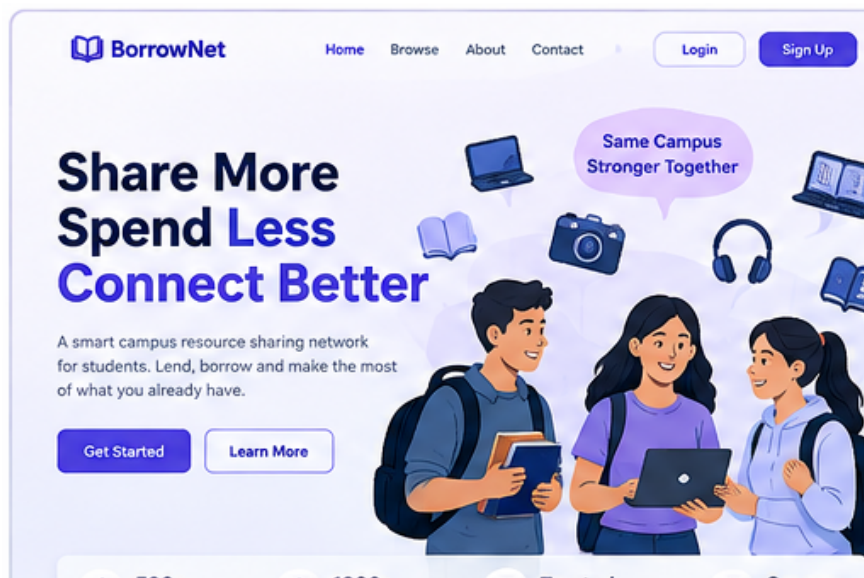


Figure E.1: Home page

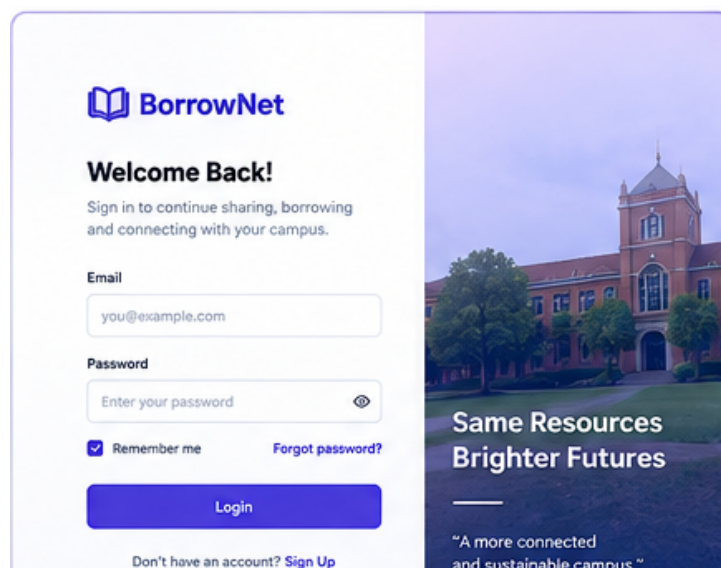


Figure E.2: Login / Registration page

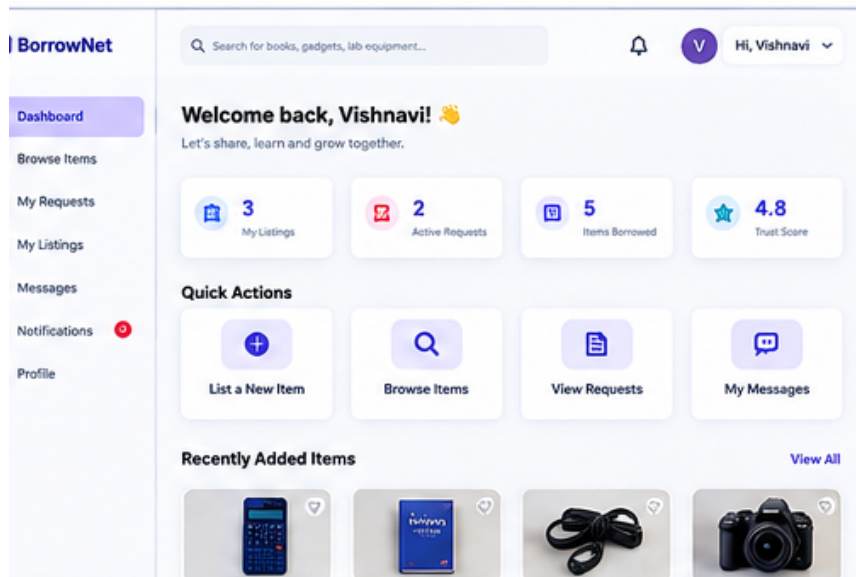


Figure E.3: Student dashboard

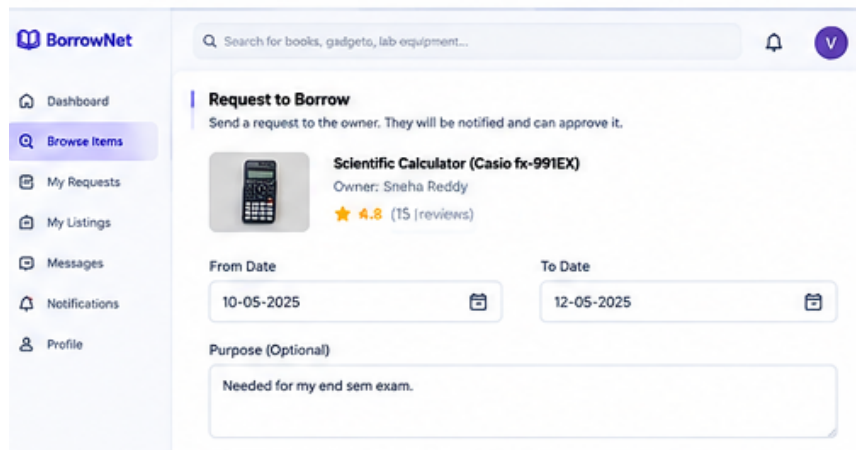


Figure E.4: Borrow request form